Combinatorial Problem Solving
Integer Linear Programming report

Luis Fernando Chavez Quevedo

May 27, 2026

GitHub repository available at https://github.com/Luisfc68/combinatorial-problem-solving

1 Problem summary

The problem considers a set of streets in which some streets are already one-way streets and some others are two-way streets. The goal is to free space that can later be used for parking by converting as many two-way streets as possible into one-way streets.

However, this conversion is subject to the following restriction: if, in the original configuration, there exists a path from one intersection to another, then such reachability must be preserved after the modifications. Moreover, the cost of the new shortest path between any such pair of intersections cannot increase by more than a given threshold percentage with respect to the original shortest-path cost.

The instance is represented by a cost matrix, where each entry indicates the travel cost from one intersection to another. The solution consists of the selected street orientations, represented as ordered pairs, together with the total number of streets converted into one-way streets.

2 Variables and constraints used

2.1 Decision Variables

Let n be the number of crossings, identified with $\{0, 1, \ldots, n-1\}$.

2.1.1 Street orientation

For each pair of crossings (i,j) with $0 \leq i,j < n$, we define the binary variable

$$x_{ij} = \begin{cases} 1 & \text{if arc } i \to j \text{ remains active after the changes,} \\ 0 & \text{otherwise.} \end{cases}$$

In the code this corresponds to the `stays` array, where `cell(stays, n, i, j)` represents x_{ij}.

2.1.2 Path flow

For each ordered pair of crossings (s,t) and for each arc (k,l), with $0 \leq s,t,k,l < n$, we define the binary variable

$$f(s,t)_{kl} = \begin{cases} 1 & \text{if arc } k \to l \text{ is part of the path from } s \text{ to } t, \\ 0 & \text{otherwise.} \end{cases}$$

In the code this corresponds to `paths[s][t]`, where `cell(paths[s][t],n,k,l)` represents $f(s,t)_{kl}$. This means that each element in the path matrix is a matrix itself of the same size as the original matrix.

2.2 Parameters and Sets

- t_{ij}: travel time from crossing i to crossing j in the original configuration ($t_{ij} = -1$ if no direct arc exists, $t_{ii} = 0$).
- D_{ij}: length of the shortest path from i to j in the original network, precomputed with Floyd-Warshall ($D_{ij} = -1$ if j is unreachable from i).
- p: threshold percentage.
- $E = \{(i,j) : t_{ij} \neq -1,\ i \neq j\}$: arcs present in the original graph.
- $O = \{(i,j) : t_{ij} \neq -1,\ t_{ji} = -1\}$: one-way streets.
- $T = \{\{i,j\} : i < j,\ t_{ij} \neq -1,\ t_{ji} \neq -1\}$: two-way streets.
- $R = \{(s,t) : s \neq t,\ D_{st} \neq -1\}$: originally reachable ordered pairs.

2.3 Constraints

(C1) One-way streets are fixed. One-way arcs cannot be changed, so their orientation is forced to remain active:

$$x_{ij} = 1 \qquad \forall (i,j) \in O.$$

(C2) Non-existent arcs stay disabled. No new streets can be built:

$$x_{ij} = 0 \qquad \forall (i,j) | t_{ij} = -1.$$

(C3) Two-way streets are not destroyed. At least one direction of every two-way street must survive:

$$x_{ij} + x_{ji} \geq 1 \qquad \forall \{i,j\} \in T.$$

(C4) Flow uses only active arcs. For every reachable pair, flow may only travel along arcs that remain active. This means that the variable of the arc (k,l) in the flow (s,t) ($f(s,t)_{kl}$) can only be active if the arc (k,l) remains active in the solution:

$$f(s,t)_{kl} \leq x_{kl} \qquad \forall (s,t) \in R,\ \forall (k,l) \in \{0,\ldots,n-1\}.$$

(C5) Flow conservation. For every reachable pair (s,t), the flow forms a path from s to t. In other words, for each vertex v in the (s,t) path, the in flow and the out flow must be the same, unless $v = s$ or $v = t$, because if is the origin of the path there is no in flow as it is the source and if is the end there is no out flow as it is the sink:

$$\sum_{l=0}^{n-1} f(s,t)_{vl} \ - \ \sum_{l=0}^{n-1} f(s,t)_{lv} = \begin{cases} 1 & \text{if } v = s, \\ -1 & \text{if } v = t, \\ 0 & \text{otherwise,} \end{cases} \qquad \forall (s,t) \in R,\ \forall v \in \{0,\ldots,n-1\}.$$

(C6) Time tolerance. For every reachable pair, the cost of the chosen path may not exceed the original distance by more than $p\%$:

$$\sum_{(k,l)\in E} t_{kl}\, f(s,t)_{kl} \;\le\; \left(1+\tfrac{p}{100}\right) D_{st} \qquad \forall (s,t)\in R.$$

Constraints C5 and C6 do not force $f(s,t)$ to coincide with the shortest path from s to t, they only require the existence of some s-t path whose cost is at most $\left(1+\frac{p}{100}\right) D_{st}$. This is sufficient because the shortest-path cost lower-bounds the cost of any s-t path, the existence of a feasible path within the tolerance implies that the true shortest path also respects it. Therefore we don't need to find the shortest path, just by proving there is a path within the tolerance is enough.

2.4 Objective function

Maximize the number of two-way streets converted to one-way:

$$\max \sum_{\{i,j\}\in T} \left(2 - x_{ij} - x_{ji}\right).$$

Each two-way street contributes 0 if it stays two-way ($x_{ij} = x_{ji} = 1$) and 1 if it is converted to one-way (exactly one orientation survives). Constraint C3 prevents the case where both directions are removed, which would contribute 2. Therefore, maximizing the sum maximizes the number of conversions.

3 Implementation details

In contrast with the constraint programming approach, the integer linear programming approach didn't require to implement any special customization to solve the problem. The implication of this was that there was more time spent on designing the solution rather than implementing it. In any case, there are a few implementation details worth mentioning:

- **Floyd-Warshall usage** in the same way it was taken from the checker source code in the constraint programming approach, here it was also used to compute the matrix D with the lengths of the shortest paths. The difference is that contrary to the other implementation, here it didn't have to be called multiple times, just at the beginning of the program.
- **Multiple loops in model creation** looping through the different pairs to create the full model can be done only once, however, the implementation does it multiple times in order to improve the separation and therefore the readability of the constraints. This doesn't represent an important impact in the performance as its only done during the model creation.
- **Avoided creation of unnecessary variables** even though it is defined in section 2.1.2 that each pair has its own matrix to represent the flow between its vertices, in the actual implementation we don't create this matrices in the pairs that are unreachable in the original graph nor in the pairs that are part of the diagonal of the matrix. The purpose of this is to avoid creating additional variables that are not required for the solution.

4 Comparison against CP

The purpose of this section is to analyze the difference between the results obtained with the constraint programming implementation and the integer linear programming implementation.

The benchmark consists of 111 instances ranging from 4 to 12 crossings. Since all test instances are relatively small, the runtime and memory figures reported below should be read considering the asymptotic $\mathcal{O}(n^4)$ growth of the ILP model. The observed advantage of CP is consistent with this growth because as the number of crossings increases, the ILP is expected to degrade faster than the CP model, so the gap measured on these small instances should only increase on larger ones.

4.1 Results

After generating the results with each implementation in the `out` directory in each case, the comparison was carried out with `$diff -r integer-linear-programming/out/ constraint-programming/out/ > diff.txt`. The diff was non-empty, which at first suggests that the two solvers produced different solutions. However, inspecting each individual diff (see Appendix A) shows that the reported objective value (the number of streets converted to one-way) is identical in every instance; the only differences are the chosen orientation of some converted streets. Since the problem counts the number of conversions and not their direction, these differences do not affect optimality. In fact, both solvers selecting opposite orientations for the same street confirms that both orientations are feasible and optimal for it. In essence, the two approaches produce the same result.

4.2 Performance

Each solver was executed three times in containers with the same cap on the resources by using the command `$docker run -rm -cpuset-cpus="2" -cpuset-mems="0" -memory="8g" -memory-swap="8g" -v ./input:/app/input -v ./out:/app/out <name of the image> time -v /app/solve.sh sdp input out`. The full results can be found in the appendix B, however, they are summarized in the table 1:

Metric	CP	ILP	Ratio (ILP/CP)
Wall-clock time (s)	3.07	24.40	7.9×
User CPU time (s)	0.68	12.75	18.8×
System CPU time (s)	0.72	2.78	3.9×
Total CPU time (s)	1.40	15.53	11.1×
CPU utilization (%)	45	63	1.4×
Max. resident memory (MB)	7.5	74.5	9.9×
Minor page faults	68 653	396 479	5.8×
Voluntary context switches	3 909	13 804	3.5×
Involuntary context switches	76	6 796	89×

Table 1: Average resource usage over three runs per method.

The CP model outperforms the ILP across every dimension: it is about 8× faster in wall-clock time, 11× cheaper in CPU time, and 10× lighter in memory. This is consistent with the structure of the ILP, whose flow formulation introduces a variable $f(s,t)_{kl}$ for every arc and every reachable pair, producing approximately $\mathcal{O}(n^4)$ binary variables; the resulting search space results in both the longer runtime and the larger memory requirements. In short, although the ILP is correct and returns the optimum, its flow-based encoding scales poorly in time and space, suggesting CP might be a more practical choice as the number of crossings grows.

5 Conclusion

After analyzing the results and performance obtained after running the ILP based solver for the project instances we can conclude that for this particular set of instances the CP based solver is a better choice as it not only produces the same results, but it also does it in a more efficient way CPU and memory wise.

These efficiency gains, however, come with trade-offs that go beyond performance. The CP solution relied on a custom propagator, which ties it to a specific solver and makes it harder to port elsewhere. The ILP, in contrast, is a standard, self-contained formulation that can be adapted to virtually any MILP solver with little change, making it the more general and reproducible of the two, this is an advantage whenever portability or a cross-solver comparison is a goal. Neither approach is trivial to model: expressing the shortest-path tolerance using the flow network approach is not straightforward, just as the CP encoding required a dedicated propagator. Tooling is another practical factor to consider, Gecode is open source, whereas CPLEX is proprietary and license-bound, which can be decisive in a production setting.

To summarize, for the current set of instances and this problem in particular our indicators point to the CP-based solver as the more efficient option in both time and memory, while the ILP remains the more portable and general

formulation. The appropriate choice therefore depends on whether one prioritizes runtime efficiency (CP) or solver independence and ease of reuse (ILP).

A Results difference

```
diff -r integer-linear-programming/out/sdp.101.out constraint-programming/out/sdp.101.out
15,18c15,18
< 3 1
< 9 1
< 6 2
< 4 3
---
> 1 3
> 1 9
> 2 6
> 3 4
20,21c20,21
< 5 4
< 6 5
---
> 4 5
> 5 6
diff -r integer-linear-programming/out/sdp.102.out constraint-programming/out/sdp.102.out
13c13
< 7 1
---
> 1 7
diff -r integer-linear-programming/out/sdp.103.out constraint-programming/out/sdp.103.out
13,15c13,15
< 4 0
< 8 0
< 2 1
---
> 0 4
> 0 8
> 1 2
diff -r integer-linear-programming/out/sdp.104.out constraint-programming/out/sdp.104.out
12c12
< 3 1
---
> 1 3
diff -r integer-linear-programming/out/sdp.105.out constraint-programming/out/sdp.105.out
13,17c13,17
< 5 0
< 8 0
< 7 4
< 9 4
< 8 7
---
> 0 5
> 0 8
> 4 7
```

```
> 4 9
> 7 8
diff -r integer-linear-programming/out/sdp.106.out constraint-programming/out/sdp.106.out
18c18
< 9 1
---
> 1 9
diff -r integer-linear-programming/out/sdp.107.out constraint-programming/out/sdp.107.out
13c13
< 2 0
---
> 0 2
16c16
< 6 5
---
> 5 6
diff -r integer-linear-programming/out/sdp.108.out constraint-programming/out/sdp.108.out
13c13
< 5 0
---
> 0 5
diff -r integer-linear-programming/out/sdp.109.out constraint-programming/out/sdp.109.out
14c14
< 7 0
---
> 0 7
18,20c18,20
< 4 2
< 5 2
< 8 3
---
> 2 4
> 2 5
> 3 8
22,25c22,25
< 7 5
< 9 7
< 9 8
< 10 9
---
> 5 7
> 7 9
> 8 9
> 9 10
diff -r integer-linear-programming/out/sdp.11.out constraint-programming/out/sdp.11.out
10c10
< 3 0
---
> 0 3
13c13
< 3 2
---
> 2 3
diff -r integer-linear-programming/out/sdp.110.out constraint-programming/out/sdp.110.out
```

```
17,18c17,18
< 9 0
< 7 1
---
> 0 9
> 1 7
20c20
< 7 3
---
> 3 7
diff -r integer-linear-programming/out/sdp.12.out constraint-programming/out/sdp.12.out
10,12c10,12
< 5 0
< 4 1
< 5 4
---
> 0 5
> 1 4
> 4 5
diff -r integer-linear-programming/out/sdp.14.out constraint-programming/out/sdp.14.out
10c10
< 3 0
---
> 0 3
12c12
< 6 2
---
> 2 6
diff -r integer-linear-programming/out/sdp.16.out constraint-programming/out/sdp.16.out
10,11c10,11
< 3 0
< 3 1
---
> 0 3
> 1 3
diff -r integer-linear-programming/out/sdp.18.out constraint-programming/out/sdp.18.out
10,11c10,11
< 2 1
< 3 1
---
> 1 2
> 1 3
diff -r integer-linear-programming/out/sdp.21.out constraint-programming/out/sdp.21.out
10c10
< 6 0
---
> 0 6
diff -r integer-linear-programming/out/sdp.23.out constraint-programming/out/sdp.23.out
8,10c8,10
< 2 0
< 3 1
< 4 1
---
> 0 2
```

```
> 1 3
> 1 4
diff -r integer-linear-programming/out/sdp.24.out constraint-programming/out/sdp.24.out
11,12c11,12
< 4 2
< 2 5
---
> 2 4
> 5 2
diff -r integer-linear-programming/out/sdp.26.out constraint-programming/out/sdp.26.out
10c10
< 5 0
---
> 0 5
13c13
< 5 3
---
> 3 5
diff -r integer-linear-programming/out/sdp.3.out constraint-programming/out/sdp.3.out
10c10
< 4 2
---
> 2 4
diff -r integer-linear-programming/out/sdp.32.out constraint-programming/out/sdp.32.out
11c11
< 4 2
---
> 2 4
diff -r integer-linear-programming/out/sdp.37.out constraint-programming/out/sdp.37.out
8,9c8,9
< 3 0
< 3 2
---
> 0 3
> 2 3
diff -r integer-linear-programming/out/sdp.39.out constraint-programming/out/sdp.39.out
10c10
< 2 0
---
> 0 2
diff -r integer-linear-programming/out/sdp.4.out constraint-programming/out/sdp.4.out
13,15c13,15
< 4 1
< 4 3
< 5 4
---
> 1 4
> 3 4
> 4 5
diff -r integer-linear-programming/out/sdp.40.out constraint-programming/out/sdp.40.out
10,11c10,11
< 4 1
< 5 2
---
```

```
> 1 4
> 2 5
diff -r integer-linear-programming/out/sdp.45.out constraint-programming/out/sdp.45.out
9c9
< 4 2
---
> 2 4
diff -r integer-linear-programming/out/sdp.52.out constraint-programming/out/sdp.52.out
10c10
< 1 0
---
> 0 1
12c12
< 4 1
---
> 1 4
diff -r integer-linear-programming/out/sdp.53.out constraint-programming/out/sdp.53.out
9c9
< 5 0
---
> 0 5
diff -r integer-linear-programming/out/sdp.58.out constraint-programming/out/sdp.58.out
9c9
< 2 0
---
> 0 2
diff -r integer-linear-programming/out/sdp.6.out constraint-programming/out/sdp.6.out
9,10c9,10
< 4 0
< 4 1
---
> 0 4
> 1 4
diff -r integer-linear-programming/out/sdp.60.out constraint-programming/out/sdp.60.out
11c11
< 6 1
---
> 1 6
13c13
< 5 4
---
> 4 5
diff -r integer-linear-programming/out/sdp.61.out constraint-programming/out/sdp.61.out
8c8
< 2 0
---
> 0 2
diff -r integer-linear-programming/out/sdp.71.out constraint-programming/out/sdp.71.out
11c11
< 2 0
---
> 0 2
13,14c13,14
< 5 1
```

```
< 6 2
---
> 1 5
> 2 6
diff -r integer-linear-programming/out/sdp.72.out constraint-programming/out/sdp.72.out
11c11
< 6 5
---
> 5 6
diff -r integer-linear-programming/out/sdp.74.out constraint-programming/out/sdp.74.out
15c15
< 4 2
---
> 2 4
diff -r integer-linear-programming/out/sdp.75.out constraint-programming/out/sdp.75.out
8,9c8,9
< 1 0
< 4 0
---
> 0 1
> 0 4
diff -r integer-linear-programming/out/sdp.78.out constraint-programming/out/sdp.78.out
12c12
< 6 3
---
> 3 6
diff -r integer-linear-programming/out/sdp.81.out constraint-programming/out/sdp.81.out
10c10
< 3 1
---
> 1 3
diff -r integer-linear-programming/out/sdp.87.out constraint-programming/out/sdp.87.out
10,11c10,11
< 3 0
< 6 5
---
> 0 3
> 5 6
diff -r integer-linear-programming/out/sdp.89.out constraint-programming/out/sdp.89.out
10c10
< 4 3
---
> 3 4
diff -r integer-linear-programming/out/sdp.91.out constraint-programming/out/sdp.91.out
10,11c10,11
< 1 0
< 4 0
---
> 0 1
> 0 4
diff -r integer-linear-programming/out/sdp.93.out constraint-programming/out/sdp.93.out
10c10
< 2 1
---
```

```
> 1 2
diff -r integer-linear-programming/out/sdp.94.out constraint-programming/out/sdp.94.out
9c9
< 4 0
---
> 0 4
```

B Performance results

```
ILP results
 Command being timed: "/app/solve.sh sdp input out"
        User time (seconds): 12.69
        System time (seconds): 2.79
        Percent of CPU this job got: 63%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:24.45
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 76156
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 185
        Minor (reclaiming a frame) page faults: 396162
        Voluntary context switches: 13853
        Involuntary context switches: 6707
        Swaps: 0
        File system inputs: 31648
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0




        Command being timed: "/app/solve.sh sdp input out"
        User time (seconds): 12.63
        System time (seconds): 2.79
        Percent of CPU this job got: 63%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:24.18
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 75776
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 396262
        Voluntary context switches: 13776
        Involuntary context switches: 6790
        Swaps: 0
```

```
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0


        Command being timed: "/app/solve.sh sdp input out"
        User time (seconds): 12.92
        System time (seconds): 2.76
        Percent of CPU this job got: 63%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:24.58
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 77056
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 397013
        Voluntary context switches: 13782
        Involuntary context switches: 6892
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0

CP results
        Command being timed: "/app/solve.sh sdp input out"
        User time (seconds): 0.69
        System time (seconds): 0.74
        Percent of CPU this job got: 46%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:03.10
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 7680
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 69153
        Voluntary context switches: 3947
        Involuntary context switches: 83
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
```

```
Page size (bytes): 4096
Exit status: 0



Command being timed: "/app/solve.sh sdp input out"
User time (seconds): 0.66
System time (seconds): 0.72
Percent of CPU this job got: 45%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:03.04
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 7680
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 68951
Voluntary context switches: 3947
Involuntary context switches: 68
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0


Command being timed: "/app/solve.sh sdp input out"
User time (seconds): 0.69
System time (seconds): 0.71
Percent of CPU this job got: 45%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:03.08
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 7680
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 67854
Voluntary context switches: 3834
Involuntary context switches: 76
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```